Pensamento Computacional
INF05008

Aula 1 – Introdução

Prof. Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul

Instituto de Informática

Departamento de Informática Teórica

Slides baseados no material da Prof. Leila Ribeiro.

UFRGS

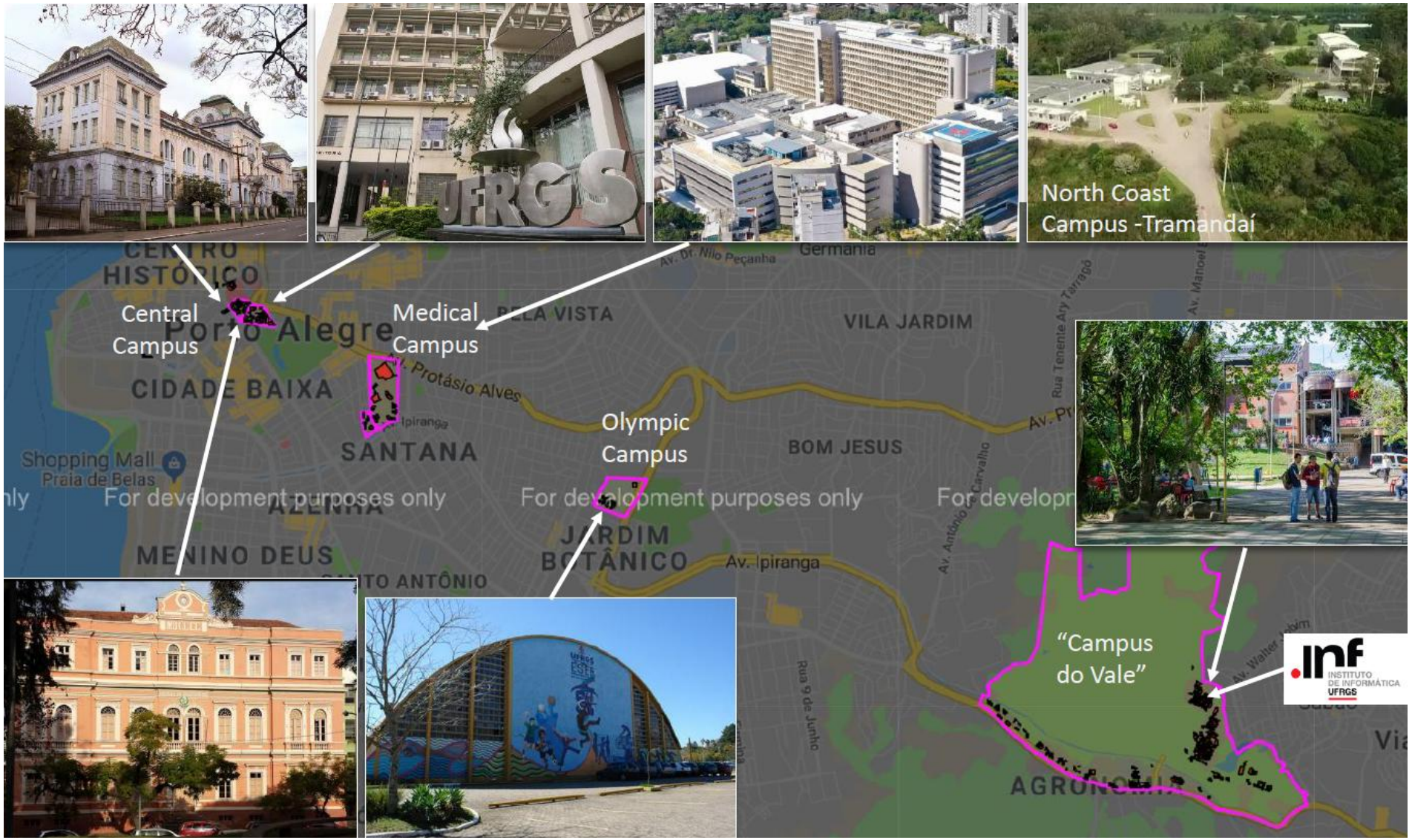
- Fundada em 1934 (primeiras faculdades datam de 1895)
- Melhor Universidade Federal do País segundo Ministério da Educação
- Top 10 melhores universidades na América Latina segundo Times Higher Education





UFRGS
UNIVERSIDADE FEDERAL
DO RIO GRANDE DO SUL

Campus em Porto Alegre



Instituto de Informática (INF)

- Um dos maiores institutos de Computação do Brasil
- ~70 docentes

- Graduação (>1200 alunos)
  - Ciência da Computação
  - Engenharia da Computação
- Pós-Graduação
  - Computação
  - Microeletrônica
  - Informática na Educação




UFRGS
ENGENHARIA DE COMPUTAÇÃO
PUMA




fundoamanhã
AI











Alumni

Muitos ex-alunos do INF hoje estão em grandes empresas de tecnologia.























INF05008
Pensamento Computacional

Professores – INF05008

- Leila Ribeiro - leila@inf.ufrgs.br
  - ➞ Doutora pela TU Berlin/Alemanha
- Lucio Mauro Duarte - lmduarte@inf.ufrgs.br
  - ➞ Doutor pelo University of London/Inglaterra
- Lucas Nunes Alegre - lnalegre@inf.ufrgs.br
  - ➞ Doutor pelo INF/UFRGS com período na Vrije Universiteit Brussel/Bélgica
- Pedro Colle – phbcolle@inf.ufrgs.br
  - ➞ Mestrando no PPGC – INF/UFRGS

Turmas

Sala de Aula 109 - 43425

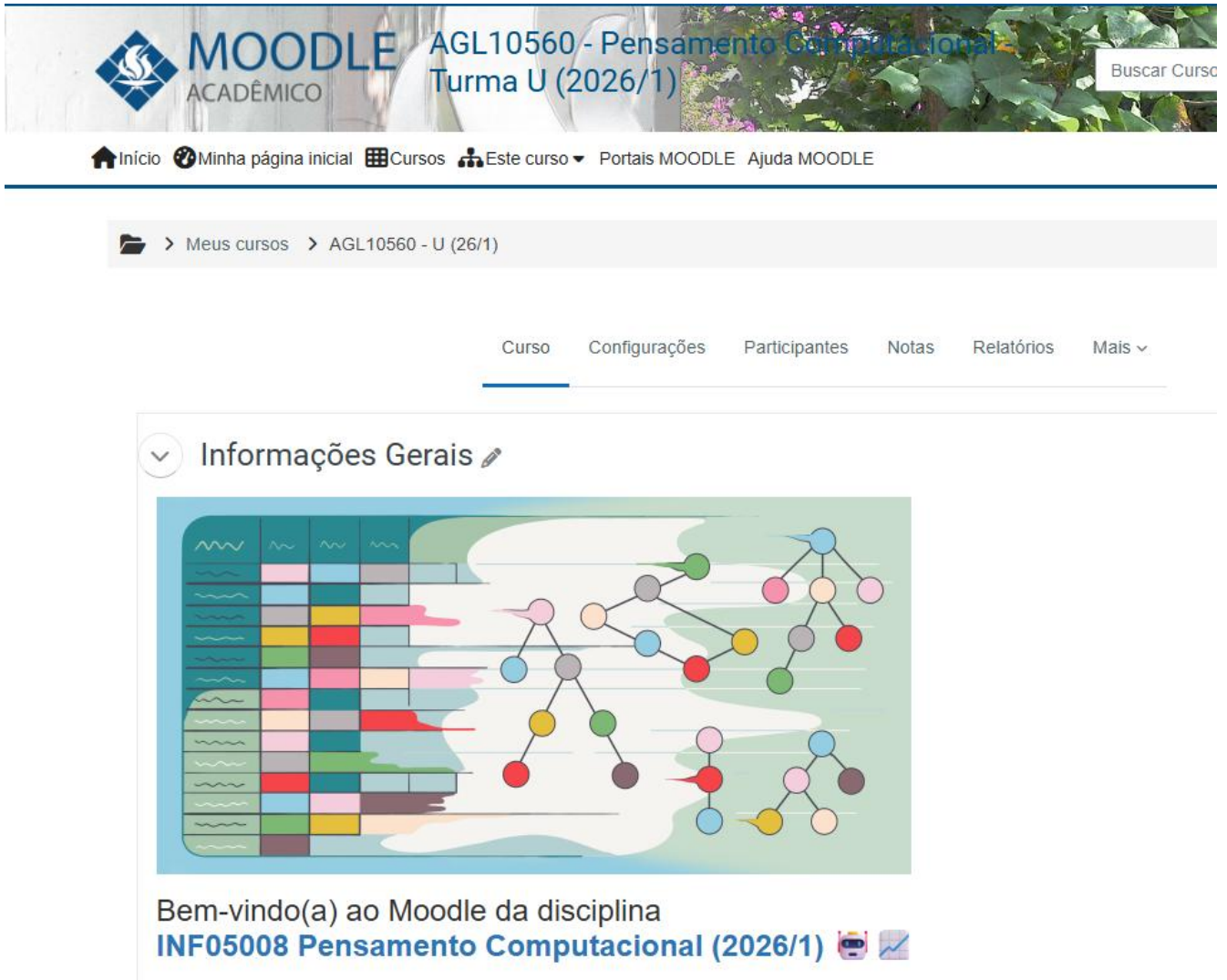
- Turma A (ter/qui 08:30) – Prof. Leila Ribeiro
- Turma B (ter/qui 10:30) – Prof. Lucio Mauro Duarte
- Turma C (ter/qui 13:30) – Prof. Lucas Nunes Alegre
- Turma D (ter/qui 15:30) – Prof. Pedro Colle

Aulas

- As aulas presenciais ocorrerão na sala 109 do prédio 43425, exceto as aulas de laboratório
- No plano de ensino estão previstas aulas de laboratório
  - salas serão divulgadas no cronograma no Moodle
- As atividades autônomas serão realizadas pelos alunos de forma individual e assíncrona
- O cronograma se encontra no Moodle da disciplina

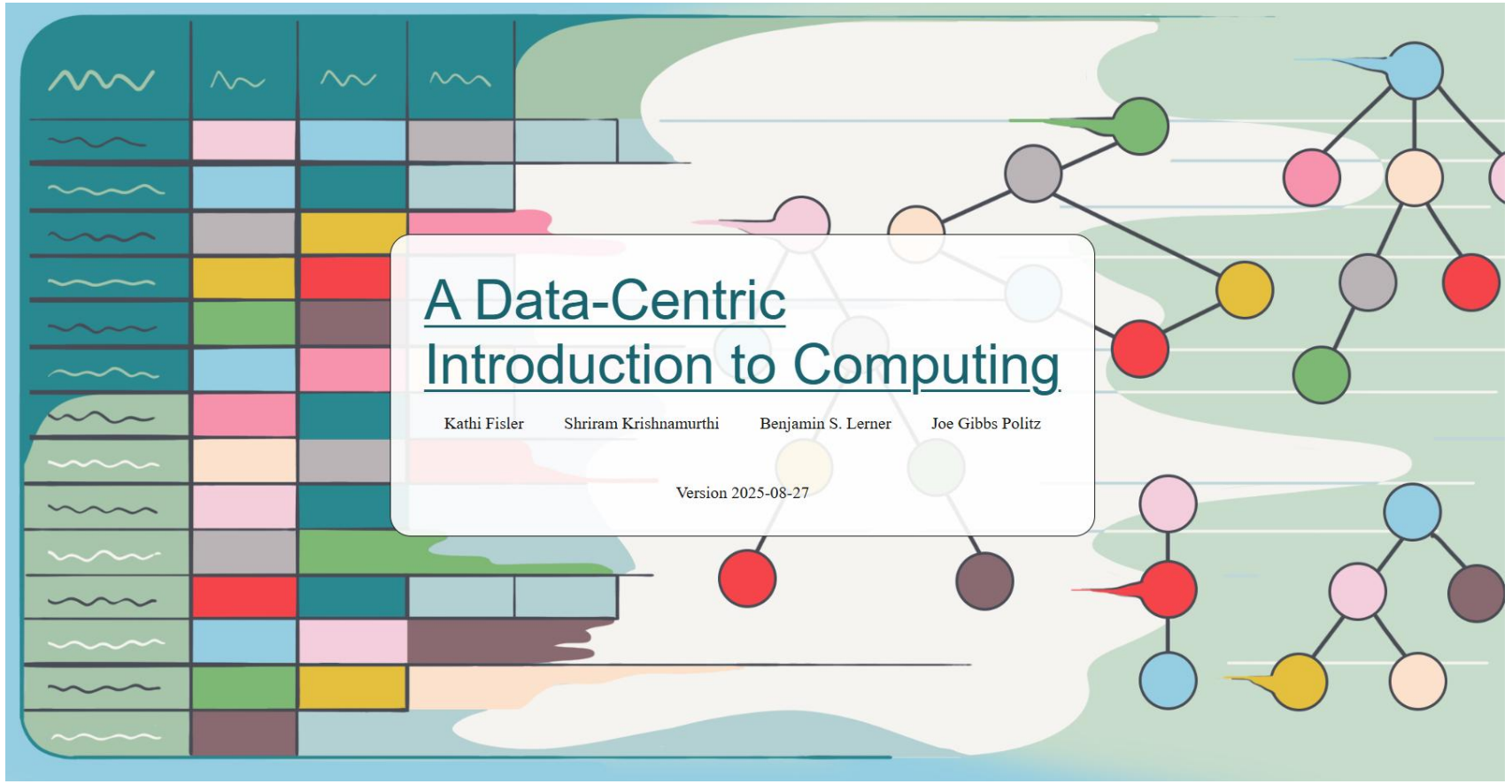
Moodle

- Moodle (UFRGS)



Bibliografia

- Livro Online: https://dcic-world.org

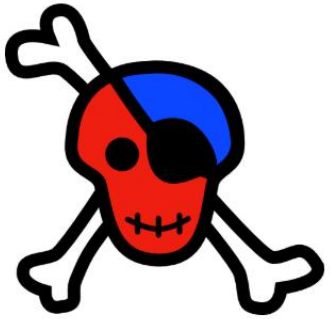


Ambientes de Programação

Pyret Interactive Editor
for VSCode



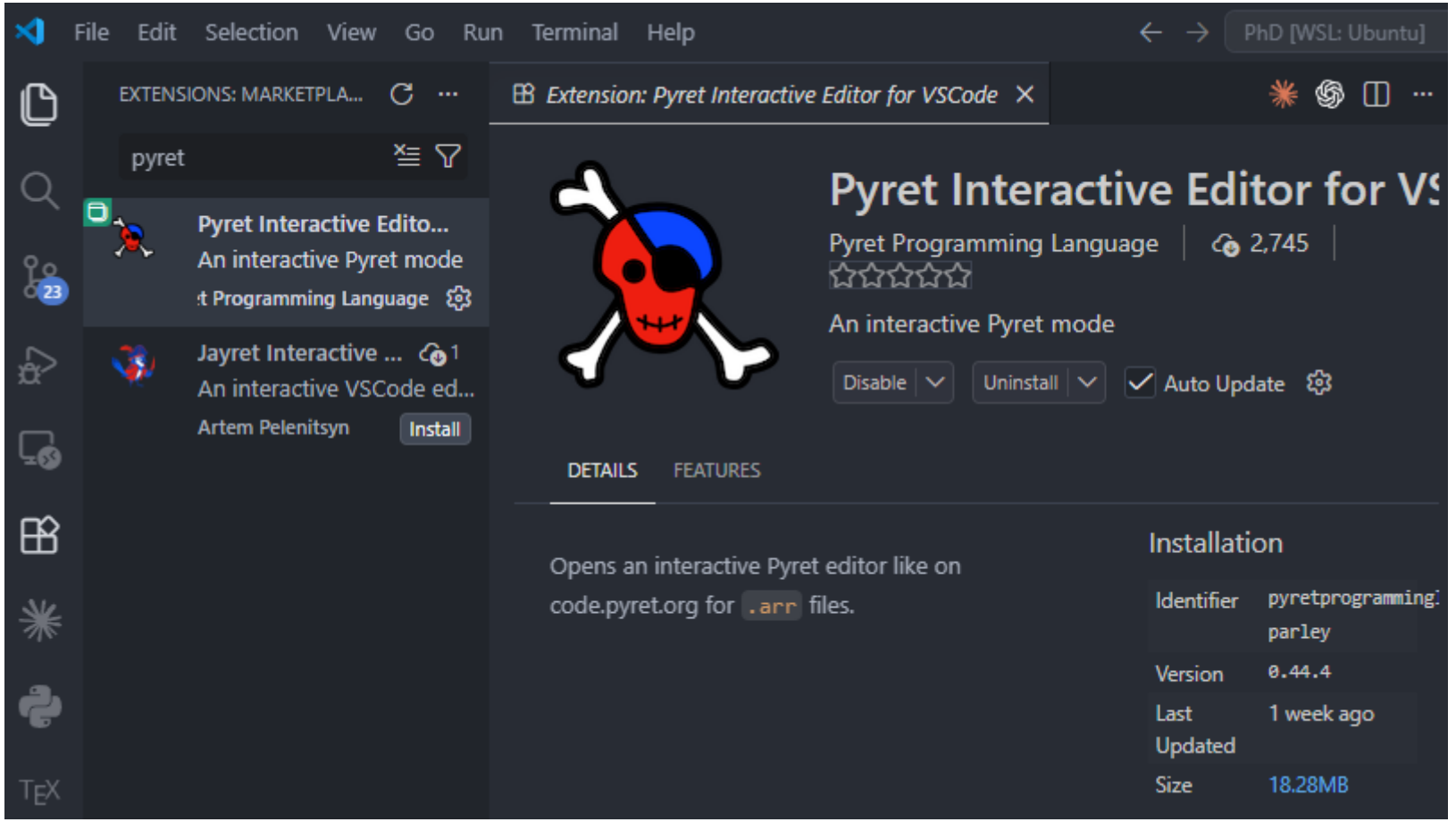
https://code.pyret.org



Open Editor
to start coding immediately

Sign in
to save and view programs

VSCode – Pyret Extension



- Instale a extensão do Pyret no seu VSCode.

Linguagem de Programação: Pyret

- Documentação da Linguagem: https://pyret.org/docs/latest/index.html

- Por quê Pyret? - https://pyret.org/pyret-code/
  - Criada para ser uma introdução a computação
  - Inspirada em outras linguagens funcionais (Scheme, Racket)
  - Sintaxe simples próxima da sintaxe de Python
  - De Pyret a Python:

  https://dcic-world.org/2025-08-27/booklet_pyret-to-python.html

- Extensão sufixo ".arr"
  - http://dialectblog.com/2011/05/24/pirate-accent/



Avaliação

Serão realizadas três provas, P1, P2 e P3. A média final (M) será calculada da seguinte forma:

$$M = P1 * 0.2 + P2 * 0.3 + P3 * 0.3 + AA * 0.2,$$

onde AA corresponde à avaliação das atividades autônomas (labs e questionários) a serem realizadas pelos alunos.

Cada atividade é avaliada individualmente e o valor AA representa a média de aproveitamento do aluno na realização destas tarefas.

A conversão da média para conceitos será realizada como descrito a seguir:

- Faltas $> 25\%$: FF (reprovado)
- $0 \leq M < 6.0$: sem conceito (recuperação) ou D - ver Ativ. de recuperação prevista
- $6.0 \leq M < 7.5$: C (aprovado)
- $7.5 \leq M < 9.0$: B (aprovado)
- $9.0 \leq M$: A (aprovado)

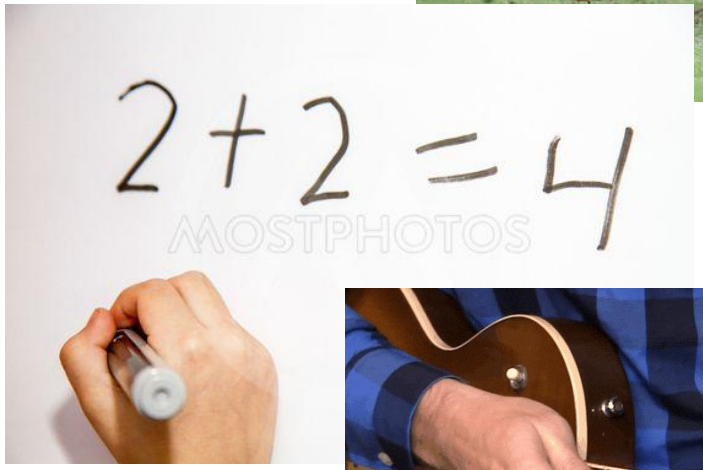
Considerações
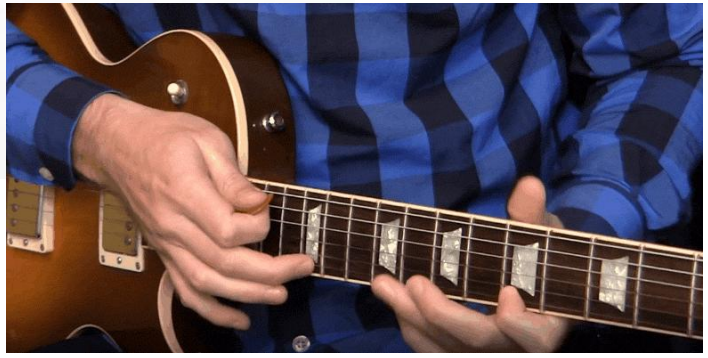
- O aluno lidera o processo de seu aprendizado.
- É importante usar o Fórum de Dúvidas da disciplina.
- É importante usar o realizar as atividades propostas na disciplina semanalmente.
- É necessário estar atento às comunicações via email e acompanhar os avisos no Moodle.

Aprendizagem na Era da IA

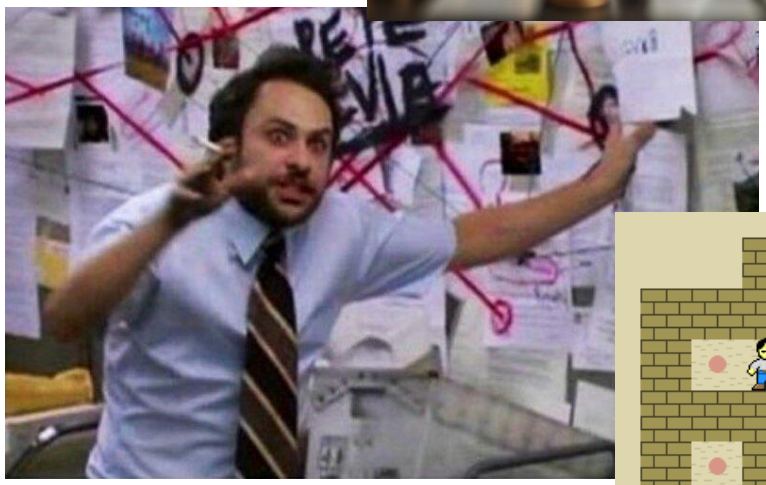


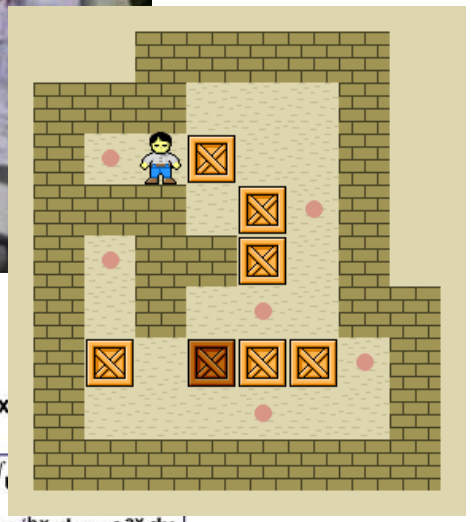












Let $z = a + bi$ $\int e^{zx}\,dx = \int e^{(a+bi)x}$

$$\int e^{(a+bi)x}\,dx = \int e^{ax}\cdot e^{ibx}\,dx$$

$$= \frac{1}{a}e^{ax}e^{ibx} - \frac{ib}{a}\int e^{ax}\cdot e^{ibx}\,dx$$

$u = e^{ibx}$ $dv = e^{ax}\,dx$

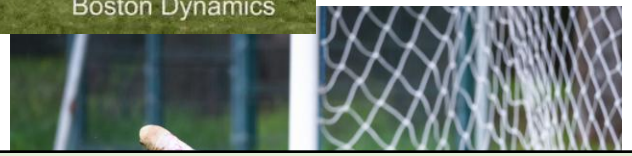
$du = ib\,e^{ibx}\,dx$ $v = \frac{1}{a}e^{ax}$

$$\left(1+\frac{ib}{a}\right)\int e^{ax}\cdot e^{ibx}\,dx = \frac{1}{a}e^{ax}e^{ibx}$$

$$\int e^{ax}\cdot e^{ibx}\,dx = \frac{\frac{1}{a}e^{ax}e^{ibx}}{\left(1+\frac{ib}{a}\right)} = \frac{e^{zx}}{a+ib} = \boxed{\frac{1}{z}e^{zx}}$$

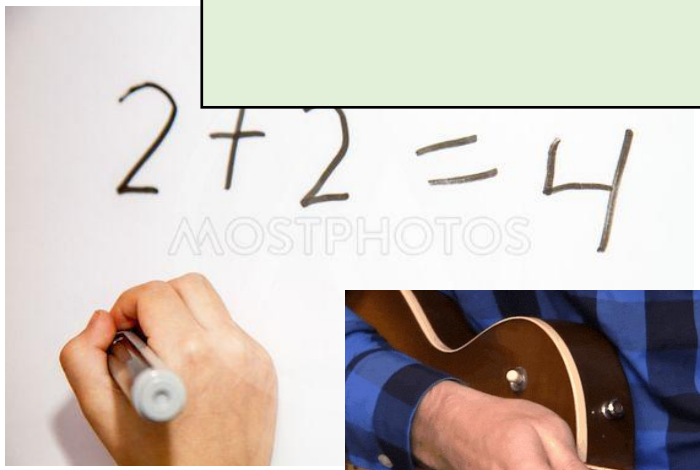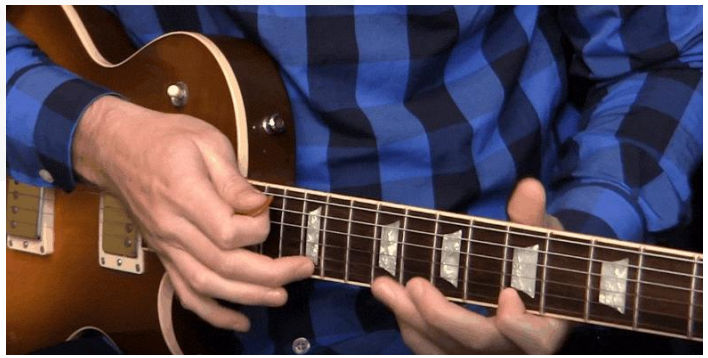








Reflexivo (automático) vs. Planejamento (raciocínio)







$$\text{Let } z = a + bi \qquad \int e^{zx}\,dx = \int e^{(a+bi)x}$$

$$\int e^{(a+bi)x}\,dx = \int e^{ax}\cdot e^{ibx}\,dx$$

$$= \frac{1}{a}e^{ax}e^{ibx} - \frac{ib}{a}\int e^{ax}\cdot e^{ibx}\,dx$$

$$u = e^{ibx} \quad dv = e^{ax}\,dx$$
$$du = ib\,e^{ibx}\,dx \quad v = \frac{1}{a}e^{ax}$$

$$\left(1+\frac{ib}{a}\right)\int e^{ax}\cdot e^{ibx}\,dx = \frac{1}{a}e^{ax}e^{ibx}$$

$$\int e^{ax}\cdot e^{ibx}\,dx = \frac{\frac{1}{a}e^{ax}e^{ibx}}{\left(1+\frac{ib}{a}\right)} = \frac{e^{zx}}{a+ib} = \boxed{\frac{1}{z}e^{zx}}$$

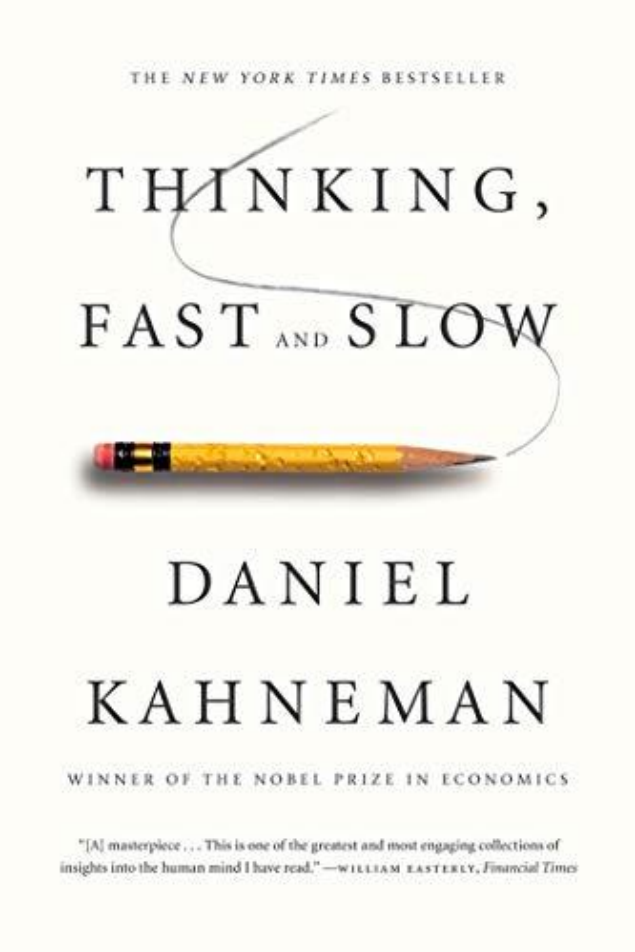
Sistema 1 vs. Sistema 2

Sistema 1

- **RÁPIDO**
- inconsciente, automático, sem esforço
- SEM autoconsciência ou controle – "O que você vê é tudo o que existe."
- **FUNÇÃO:** avalia a situação, fornece atualizações
- **98%** do nosso pensamento

Sistema 2

- **LENTO**
- deliberado e consciente, exige esforço, processo mental controlado, pensamento racional
- COM autoconsciência ou controle, lógico e cético
- **FUNÇÃO:** busca informações novas ou ausentes, toma decisões
- **2%** do nosso pensamento



Sistema 2

Sistema 1





Aprendizado do Sistema 2 se transforma em Sistema 1

Aprendizagem na Era da IA

- Provê feedback imediato
  - Extremamente útil em muitas situações!

- Reduz o esforço que é crucial para o aprendizado
  - Sistema 1 para Sistema 2







Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task△

Nataliya Kosmyna [1]
MIT Media Lab
Cambridge, MA

Eugene Hauptmann
MIT
Cambridge, MA

Ye Tong Yuan
Wellesley College
Wellesley, MA

Jessica Situ
MIT
Cambridge, MA

Xian-Hao Liao
Mass. College of Art and Design (MassArt)
Boston, MA

Ashly Vivian Beresnitzky
MIT
Cambridge, MA

Iris Braunstein
MIT
Cambridge, MA

Pattie Maes
MIT Media Lab
Cambridge, MA

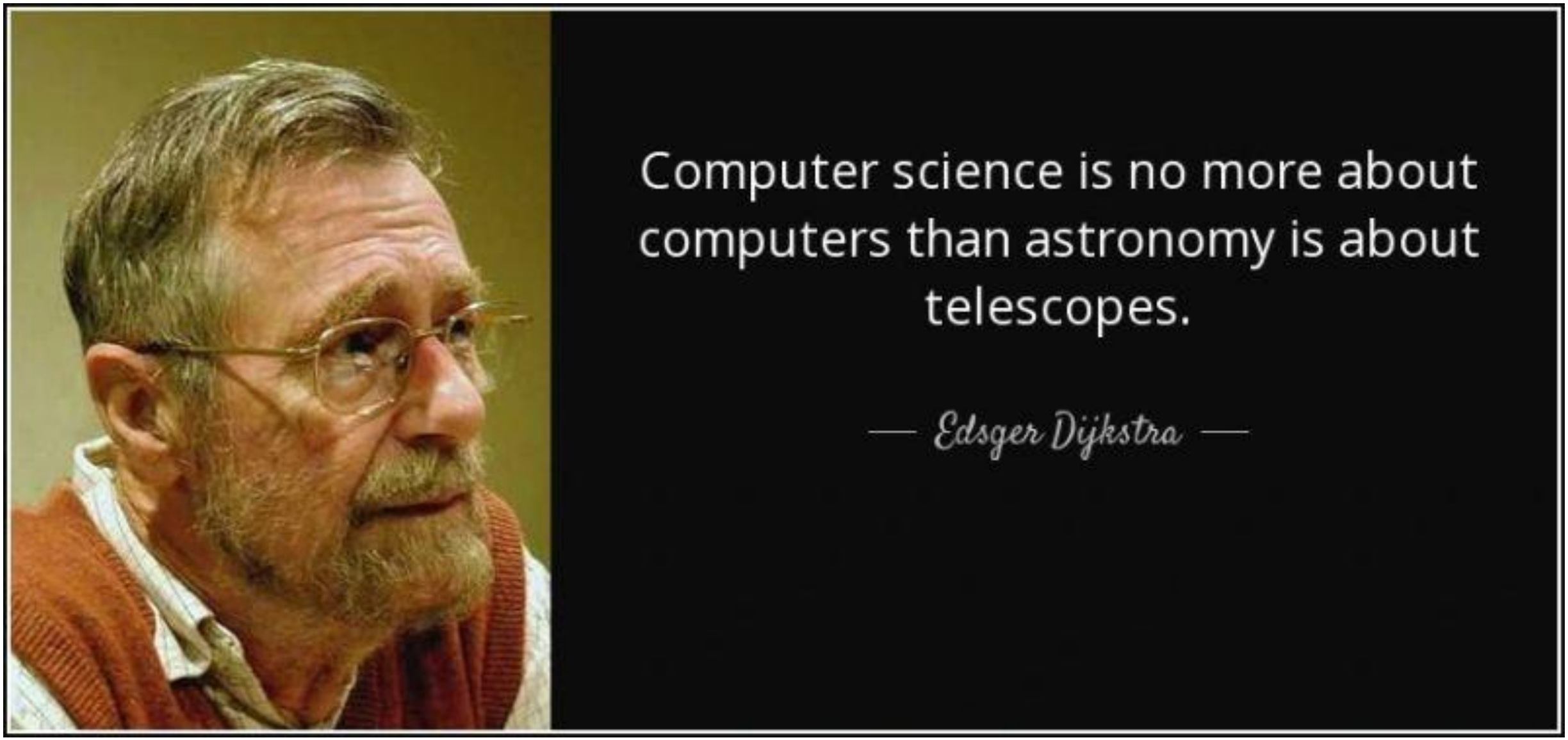
United States



Valores da Disciplina

- **Software não é escrito apenas para ser executado.**
  - Ele também precisa ser escrito para ser lido e mantido por outras pessoas.
  - Muitas vezes, essa "outra" pessoa é você mesmo, seis meses depois, que já esqueceu o que fez e por quê.
- **Os programadores são responsáveis** por garantir que seus softwares atinjam os objetivos desejados e sejam confiáveis.
  - Isso se reflete em várias disciplinas dentro da ciência da computação, como **testes** e **verificação**.
- **Os programas devem ser previsíveis.**
  - Precisamos saber, tanto quanto possível, *antes* de um programa ser executado, como ele irá se comportar.
  - Esse comportamento inclui não apenas características técnicas, como tempo de execução, uso de memória, consumo de energia, e assim por diante, mas também **impactos sociais, benefícios e danos**.

O que é Computação?


Computer science is no more about computers than astronomy is about telescopes.
— Edsger Dijkstra —

Computação

P. Se desligarmos todos os computadores do mundo, a computação deixa de existir?



- Computação existe desde antes dos computadores modernos
- A ideia formal de computação foi estudada por Alan Turing

Ideia central:

- Computação é o estudo de **processos de resolução de problemas**.

Qual o Grande Objetivo desta Disciplina?

Desenvolver o Pensamento Computacional!

Pensamento Computacional

Definição: **Pensamento Computacional** é o processo mental usado para **descrever e analisar processos**

de forma sistemática usando os

fundamentos e técnicas da ciência da computação.

Pensamento computacional envolve o domínio de

- Problemas: analisar, comparar e descrever problemas de forma precisa, objetiva e sistemática;
- Algoritmos e Modelos: criar, descrever e avaliar processos usando algoritmos e modelos computacionais.

Pensamento Computacional

O pensamento computacional desenvolve as capacidades de:

- Análise
- Automação
- Abstração

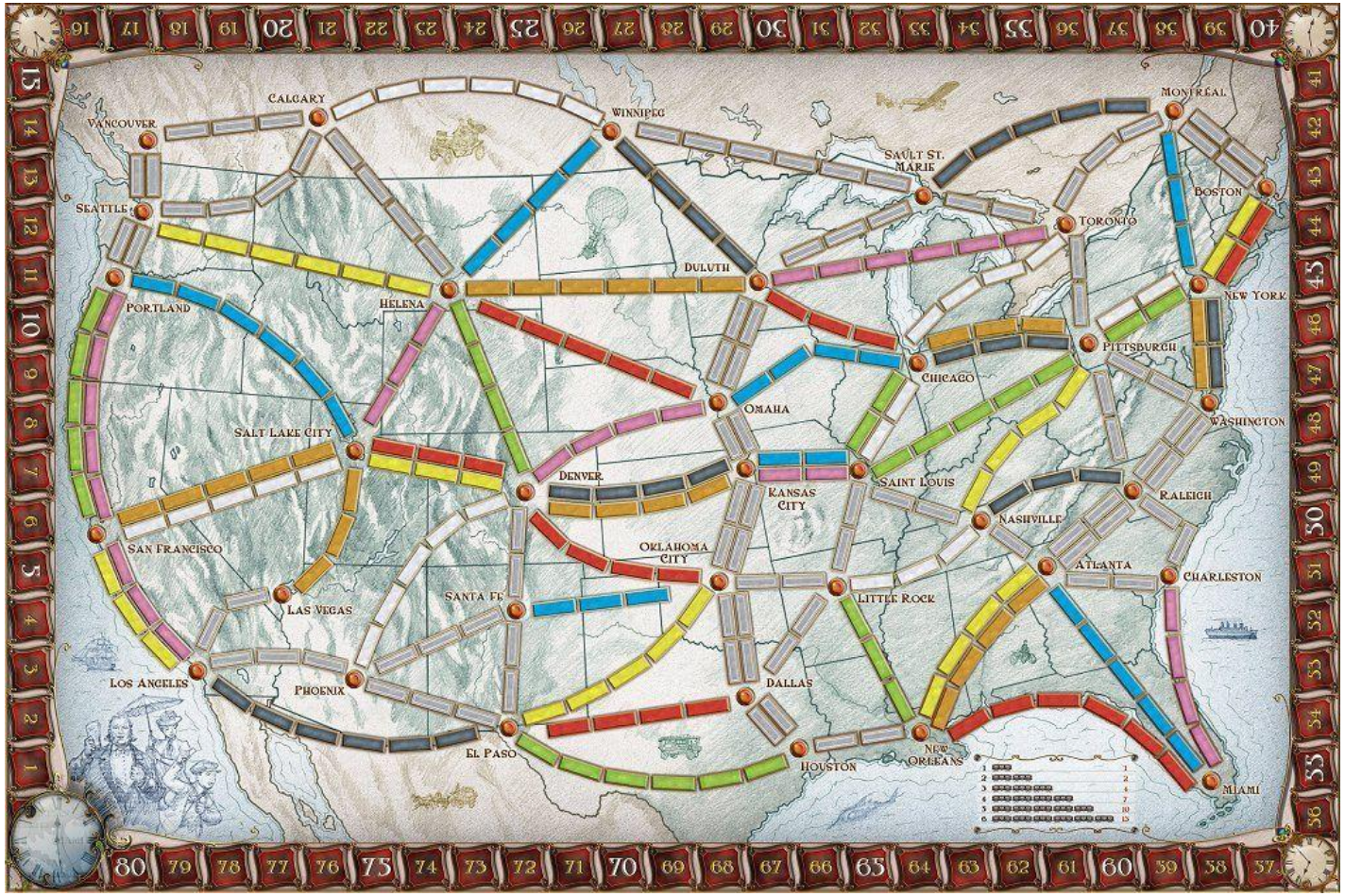
Algoritmos

- Tipicamente, expressamos soluções de problemas computacionais através de algoritmos.

- Antes de definirmos o que é um algoritmo, vamos fazer um exercício ...

Procurando Caminhos em um Mapa

- Vamos chamar o seguinte mapa de MAPA1



Tarefa 1

- No MAPA1, encontrar um caminho entre Denver e Chicago.

Caminho Denver-Chicago

Obj: Encontrar um caminho (uma lista de cidades) entre Denver e Chicago.

Entrada:

Saída: Caminho entre Denver e Chicago (lista de cidades).

P. Como generalizar?

Tarefa 2

- No MAPA1, encontrar um caminho uma cidade origem e uma cidade destino.

Caminho entre Duas Cidades

Obj: Dados uma cidade origem e uma cidade destino, encontrar um caminho (uma lista de cidades) entre origem e destino.

Entrada: Uma cidade origem e uma cidade destino.

Saída: Caminho entre cidade origem e destino (lista de cidades).

P. Como generalizar?

Tarefa 3

- Em um mapa, encontrar um caminho uma cidade origem e uma cidade destino.

Encontrar Caminho

Obj: Dados uma mapa, uma cidade origem e uma cidade destino, encontrar um caminho (uma lista de cidades) entre origem e destino.

Entrada: Um mapa, uma a cidade origem e uma cidade destino.

Saída: Caminho entre cidade origem e destino (lista de cidades).

Comparação

Caminho Denver-Chicago

Obj: Encontrar um caminho (uma lista de cidades) entre Denver e Chicago.

Entrada:

Saída: Caminho entre Denver e Chicago (lista de cidades).

Neste problema, o mapa e as cidades são fixas,
portanto a solução pode ser construída a priori.

Comparação

Encontrar Caminho

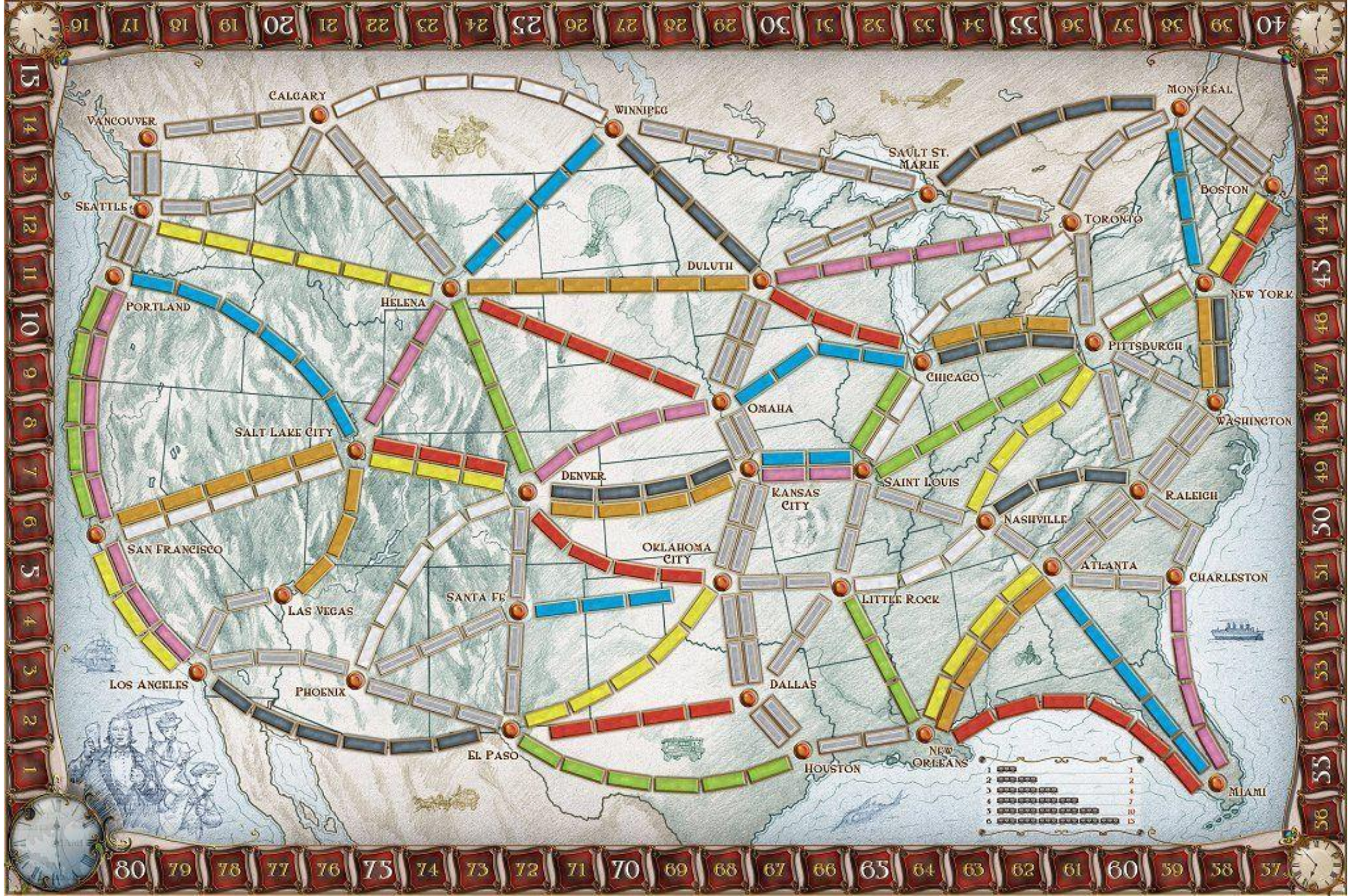
Obj: Dados uma mapa, uma cidade origem e uma cidade destino, encontrar um caminho (uma lista de cidades) entre origem e destino.

Entrada: Um mapa, uma a cidade origem e uma cidade destino.

Saída: Caminho entre cidade origem e destino (lista de cidades).

Para solucionar este problema, é necessário saber como construir um caminho.

Cada instância (mapa, origem, destino) terá uma solução diferente.


CALGARY
VANCOUVER
WINNIPEG
MONTREAL
SAULT ST. MARIE
SEATTLE
BOSTON
TORONTO
DULUTH
PORTLAND
HELENA
NEW YORK
PITTSBURGH
CHICAGO
OMAHA
WASHINGTON
SALT LAKE CITY
DENVER
SAINT LOUIS
KANSAS CITY
RALEIGH
NASHVILLE
SAN FRANCISCO
OKLAHOMA CITY
ATLANTA
CHARLESTON
SANTA FE
LITTLE ROCK
LAS VEGAS
LOS ANGELES
PHOENIX
DALLAS
EL PASO
HOUSTON
NEW ORLEANS
MIAMI

Considerações

- O que é mais fácil: encontrar um caminho ou descrever como se encontra um caminho?
- O que uma pessoa precisaria saber para conseguir entender a sua descrição (algoritmo) de como encontrar um caminho?
- Qual foi o critério usado para escolher o caminho a ser devolvido?
- O algoritmo tem algum viés?
- Existe mais de uma forma de encontrar um caminho? Se houver várias, como escolher "a melhor"?
- Você conseguiria estimar quantos caminhos existem entre Denver e Chicago (sem repetir cidades)?
- Será que conseguiríamos construir um programa que descobre os caminhos automaticamente?

Como definir “algoritmo”?

- O termo algoritmo é muito anterior à invenção do computador
- Al-Khawarizmi, matemático persa que descreveu procedimentos para resolver equações lineares
- Assim, não teria sentido falar que um algoritmo necessariamente é algo que pode ser implementado em um computador



Como definir “algoritmo”?

- Muitos definem algoritmos como “sequências de passos” para realizar algo.

- Porém, se tivermos muitas pessoas para realizar uma tarefa, várias podem executar partes da solução ao mesmo tempo. Então o algoritmo não é necessariamente uma “sequência de passos”.

- Existem inúmeras formas de descrever processos que não são baseadas em sequências de passos (usando linguagens declarativas, baseadas em eventos, etc).

Como definir “algoritmo”?

Existem características comuns a todas as definições de algoritmos:

- Um algoritmo descreve um processo
- Um algoritmo deve ser preciso
- Um algoritmo deve ser finito

Algoritmo

Definição: Algoritmo.

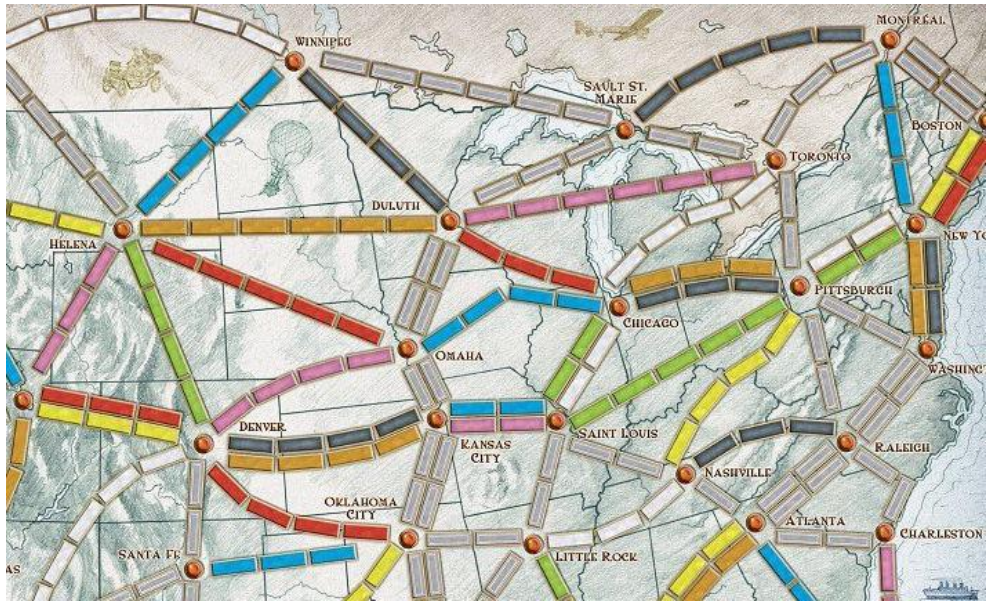
Um algoritmo é uma **descrição precisa e finita de um processo**.

Um algoritmo passível de automatização é chamado de **modelo computacional**.

Por que implementar algoritmos?

- Quantos caminhos existem neste mapa entre Denver e Chicago?

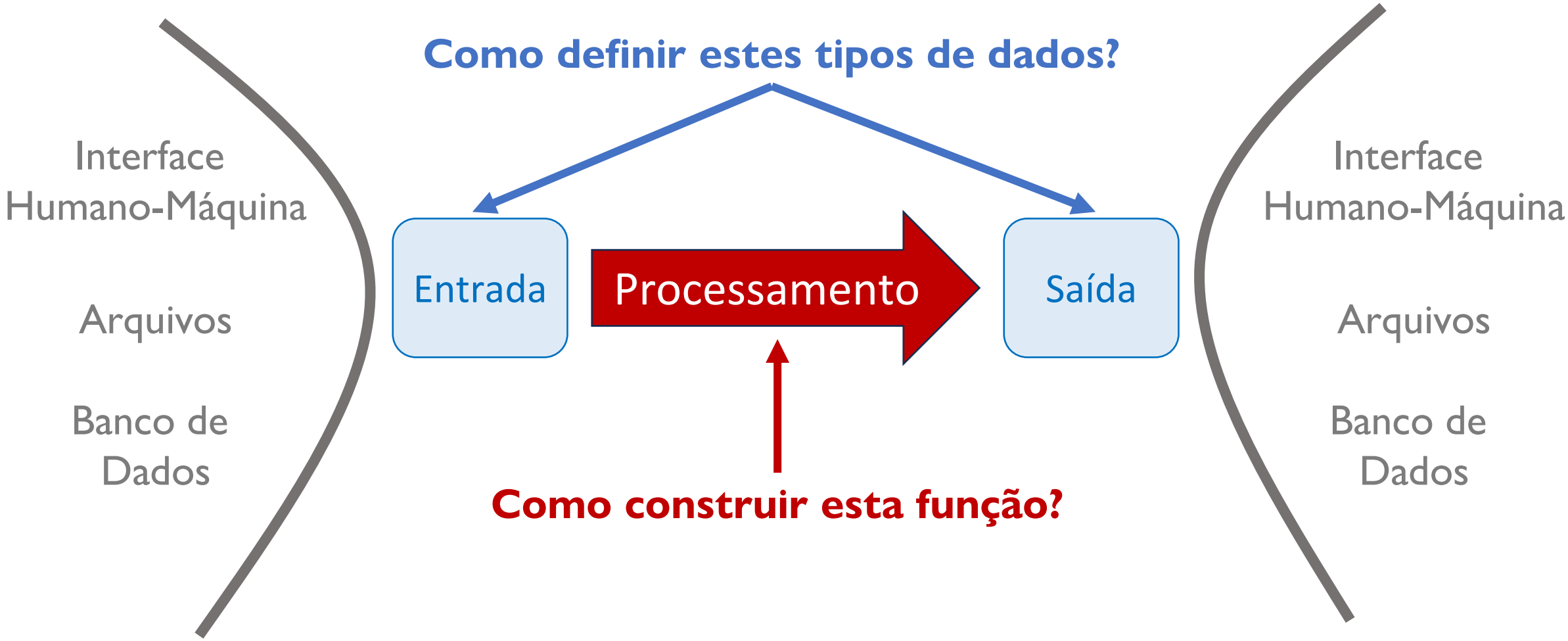
1089 caminhos!



Implementando Algoritmos

- Se conseguirmos descrever as tarefas a serem executadas com grande precisão usando linguagens adequadas, máquinas podem realizar as tarefas para nós!!!
- Computadores são máquinas que executam algoritmos!
- Podemos usar vários tipos de linguagens para nos comunicarmos com computadores, mas, em geral, precisamos construir um programa (algoritmo descrito em uma linguagem de programação)
- E os LLMs (Large Language Models)?

Foco: Pensamento Computacional



Foco da Disciplina

- Definir precisamente as soluções computacionais:
  - Entradas, saídas, documentação
- Conhecer as principais formas de estruturar dados:
  - Ex: Listas, Árvores, Grafos
- Dominar as principais técnicas de resolução de problemas:
  - Decomposição, generalização, recursão, reuso, alta ordem
- Entender e analisar programas

Tarefa de Casa

- Assistir ao vídeo sobre Introdução a Fundamentos de Algoritmos
  - https://youtu.be/Nh4lazIvuwg

- Fazer a revisão sobre Teoria dos Conjuntos
  - Ver slides no Moodle

